МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ № 3

Варіант 6

Незмінні структури даних у Python

Виконав: студент 2 курсу групи ФЕП-22
Мазан Давид

Перевірив: аспірант
Остріков Данило Андрійович

Львів - 2026

Мета роботи

Ознайомитися з незмінними структурами даних Python і застосувати їх у предметній області бронювання готелю. Перенести сутності на frozen dataclasses, tuple, frozenset і MappingProxyType, реалізувати оновлення без мутації та перевірити поведінку тестами й вимірюванням часу.

Теоретичні відомості

Незмінний об’єкт не змінюється після створення. У Python для цього можна використовувати кортежі, frozenset, dataclass з параметрами frozen=True і slots=True та MappingProxyType для доступу лише для читання до словника. Оскільки proxy є неглибоким поданням, перед його створенням треба копіювати початковий словник.

Pure update не змінює переданий запис, а створює нову версію. Для dataclass це зручно реалізувати через dataclasses.replace. Кортежі та незмінні множини дають змогу безпечно зберігати вкладені колекції. Хешування можливе, якщо всі поля, що входять до хешу, також хешовані.

Постановка завдання

За варіантом 6 реалізовано модель бронювання готелю з сутностями Guest, Service і Booking. Дані включають гостя, дати заїзду та виїзду, тариф, перелік послуг, статус, мітки й метадані. Потрібно додавати послугу, змінювати статус, продовжувати проживання, перевірити незмінність, read-only метадані, хешованість та час виконання операцій.

Хід роботи

1. Створено проєкт lab03 зі структурою, наведеною нижче. У папці report збережено цей звіт у форматах DOCX та PDF.

```
lab03/
├── report/  (lab3.docx, lab3.pdf)
├── tests/  (test_immutability.py, test_performance.py)
├── baseline.py
├── benchmark.py
├── core.py
├── main.py
├── model.py
├── README.md
├── requirements.txt
├── pyproject.toml
└── mypy.ini
```

2. У model.py визначено frozen dataclasses Guest, Service і Booking. Послуги зберігаються в tuple, мітки — у frozenset, метадані — у MappingProxyType над копією словника. Поле meta не бере участі в хеші, бо mappingproxy не хешується; решта значущих для хешу полів незмінні.

3. У core.py реалізовано чисті функції add_service, change_status, extend_stay, add_tag та update_meta. Кожна функція повертає новий об’єкт через replace. Функція booking_total обчислює вартість проживання та замовлених послуг.

4. У main.py підготовлено приклад бронювання та демонстрацію додавання послуги, продовження проживання і зміни статусу. Вхідний запис після перетворень лишається незмінним.

5. У tests/test_immutability.py перевіряються нові копії, неможливість змінити поля і вкладені колекції, read-only метадані, pure updates, валідація продовження проживання, хешованість і правильність суми. У test_performance.py перевіряються результати базової та immutable-операцій.

Результати виконання

Запуск python main.py завершився успішно. Для прикладу бронювання № 6001 створено новий запис зі статусом checked_in, трьома ночами та двома послугами. Початковий запис зберіг одну послугу.

```
=== Система бронювання готелю ===
Бронювання: #6001; гість: Олена Коваль
Статус: checked_in; ночей: 3
Послуг: 2; сума: 6050.00 грн
Початкове бронювання не змінено: 1 послуга
```

Команду pytest -q --durations=5 виконано з результатом 7 passed in 0.08s. У цьому короткому запуску окремі тривалості були меншими за поріг відображення pytest (< 0.005 s).

Мікровимірювання benchmark.py на 20 000 операцій дало 0.0368 s для mutable baseline та 0.1700 s для immutable update (приблизно 4.62 раза повільніше в цьому запуску). Це невеликий демонстраційний замір, який також включає створення прикладу для кожної операції; результати залежать від обладнання й не є універсальним висновком про продуктивність.

Перевірки mypy model.py core.py main.py baseline.py та ruff check . пройшли без зауважень. Форматування black застосовано до проєкту.

Висновок

У лабораторній роботі № 3 для варіанта 6 створено незмінну модель бронювання готелю. Кортежі, frozenset, frozen dataclasses і MappingProxyType захищають модель від випадкових змін, а чисті функції повертають нові версії запису. Усі сім автоматизованих тестів пройшли. Мікровимірювання показало вищу вартість створення immutable-оновлення для цього малого прикладу, що є очікуваним компромісом за простішу перевірку стану та відсутність прихованих побічних ефектів.